

COMPSCI 689

Lecture 25: Modeling Omnibus

Benjamin M. Marlin

College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

Memory-Based Models: KNN

- Universal function approximator closely related to kernel SVMs.
- Defined via a similarity function $S(\mathbf{x}, \mathbf{x}')$.
- Prediction for $\mathbf{x}$ is based on its K most similar neighbors in the training set as determined by $S(\mathbf{x}, \mathbf{x}')$
- Requires access to training data at test time to make predictions.

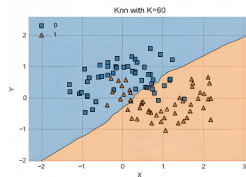
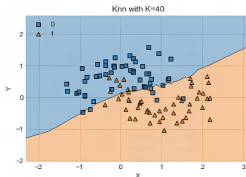
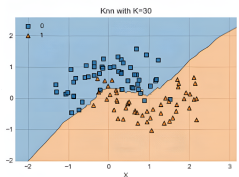
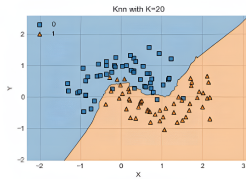
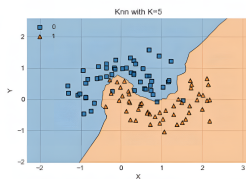
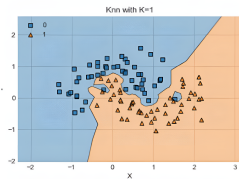
Memory-Based Models: Hard KNN

- The model has one complexity parameter, the number of neighbors K , which must be selected using a validation set.

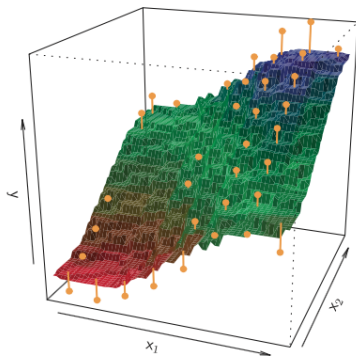
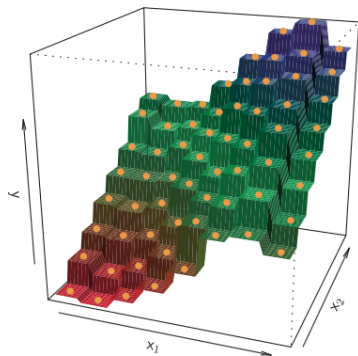
$$f(\mathbf{x}, \mathcal{D}_{tr}) = \arg \max_{c \in \mathcal{Y}} \sum_{n=1}^K [y_{\pi(n)} = c]$$
$$\pi = \arg \text{sort}([S(\mathbf{x}, \mathbf{x}_1), \dots, S(\mathbf{x}, \mathbf{x}_N)])$$

$$f(\mathbf{x}, \mathcal{D}_{tr}) = \frac{1}{K} \sum_{n=1}^K y_{\pi(n)}$$
$$\pi = \arg \text{sort}([S(\mathbf{x}, \mathbf{x}_1), \dots, S(\mathbf{x}, \mathbf{x}_N)])$$

Example: KNN Classification



Example: KNN Regression



Memory-Based Models: Weighted KNN

- Removes the need to pick a specific value of K by taking into account all data cases with similarity-derived weights.

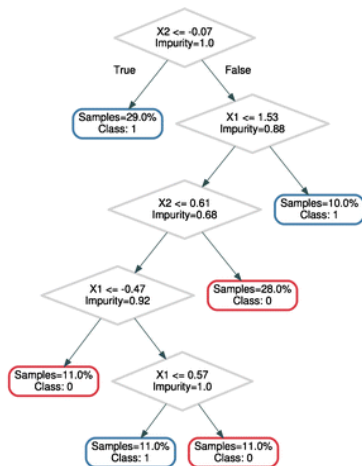
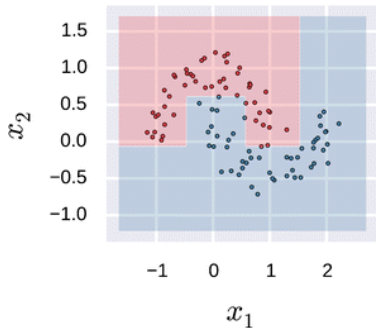
$$f(\mathbf{x}, \mathcal{D}_{tr}) = \arg \max_{c \in \mathcal{Y}} \sum_{n=1}^N w_n [y_{\pi(n)} = c], \quad w_n = \frac{S(\mathbf{x}, \mathbf{x}_n)}{\sum_{n'=1}^N S(\mathbf{x}, \mathbf{x}_{n'})}$$

$$f(\mathbf{x}, \mathcal{D}_{tr}) = \sum_{n=1}^N w_n y_n, \quad w_n = \frac{S(\mathbf{x}, \mathbf{x}_n)}{\sum_{n'=1}^N S(\mathbf{x}, \mathbf{x}_{n'})}$$

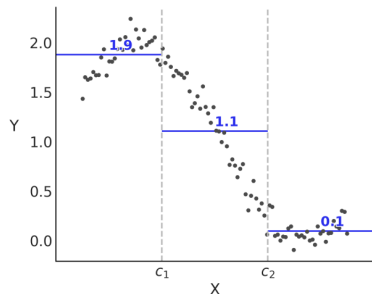
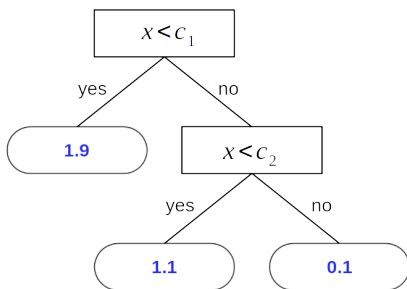
Tree-Based Models

- Tree-based models partition the feature space into a collection axis-aligned hyper-rectangular regions.
- The partitioning is defined via a nested sequence of binary splits of the feature space on the values of single variables, represented via a binary tree.
- These models assign a class label or regression function value to each region of feature space defined by the partitioning.
- In the regression case, the simplest models make piecewise constant predictions.
- The primary complexity hyper-parameter is the maximum depth of the tree.
- Shallow decision tree-based models have very strong *explainability*.

Example: Classification Tree



Example: Regression Tree



Tree-Based Models: Learning

- Given a data set, the problem of selecting a minimum loss tree of a given maximum depth is combinatorially hard.
- Instead, a greedy stage-wise algorithm is used to pick variables and split points.
- We start with all of the data assigned to the root of the tree, find the best split, then recurse on the children until the maximum depth is researched or the impurity of the node is 0.
- For a classification tree, we typically assign the majority class of the data cases assigned to a leaf as the output value for the leaf.
- For a classification tree, we typically assign the mean target value of data cases assigned to a leaf as the output value for the leaf.

Tree-Based Models: Learning

```
def getsplit(X,Y):  
    I = impurity(Y)  
    for i in range(X.shape[1]):  
        for t in X[:,i]:  
            ind = X[:,i] >= t  
            R[i,t] = I - impurity(Y[ind]) - impurity(Y[~ind])  
    i, t = argmin(R)  
    ind = X[:,i] >= t  
    return (i,t, (X[ind,:], Y[ind]), (X[~ind,:], Y[~ind]))
```

- For regression, we can use the sum of squared errors compared to the mean target value as the impurity measure.
- For classification, there are a range of different choices related to the entropy of the label target distribution.

Ensembles

- An ensemble refers to a collection of models $\mathcal{E} = \{f_{\theta_i} \mid 1 \leq i \leq M\}$ with different parameters.
- An ensemble can either be homogeneous (models all of the same type) or heterogeneous (models of different types).
- Different rules can be used to make predictions given an ensemble based on the task type.

Regression Ensembles

- In the regression setting, we typically define a prediction function for the ensemble as:

$$f_{\mathcal{E}}(\mathbf{x}) = \sum_{i=1}^M w_i \cdot f_{\theta_i}(\mathbf{x})$$

- The w_i are weights on the ensemble members that can either be set uniform (e.g., $w_i = 1/M$) or learned.

Classification Ensembles

- In the classification setting, if we only have access to an ensemble of prediction functions $f_{\theta_i}(\mathbf{x})$, we can use a prediction function based on weighted voting:

$$f_{\mathcal{E}}(\mathbf{x}) = \arg \max_{c \in \mathcal{Y}} \sum_{i=1}^M w_i \cdot [c = f_{\theta_i}(\mathbf{x})]$$

- If we have access to an ensemble of discriminant functions $g_{\theta_i}(\mathbf{x})$, we can average the weighted discriminant outputs:

$$g_{\mathcal{E}}(\mathbf{x}) = \sum_{i=1}^M w_i \cdot g_{\theta_i}(\mathbf{x})$$

Probabilistic Classification Ensembles

- If we have access to an ensemble of probabilistic classifiers $P_{\theta_i}(Y = y|\mathbf{X} = \mathbf{x})$, we can average the weighted predicted class distributions:

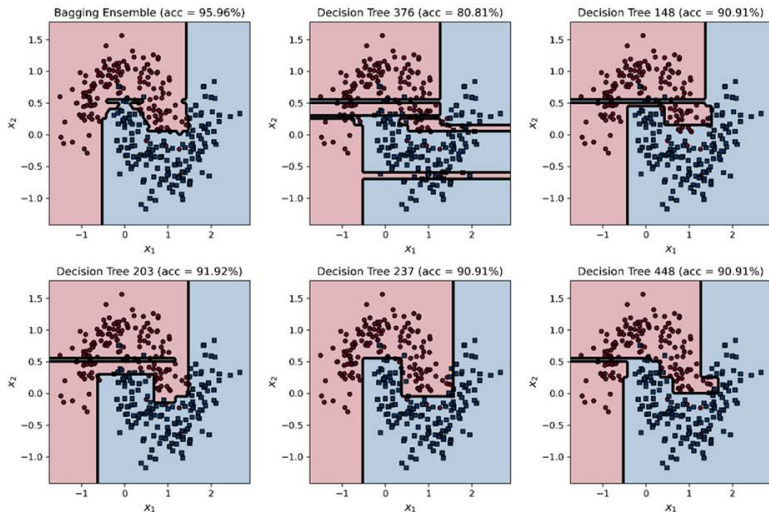
$$P_{\mathcal{E}}(Y = y|\mathbf{X} = \mathbf{x}) = \sum_{i=1}^M w_i \cdot P_{\theta_i}(Y = y|\mathbf{X} = \mathbf{x})$$

- In this case, to ensure $P_{\mathcal{E}}(Y = y|\mathbf{X} = \mathbf{x})$ is a valid mass function, it suffices for the weights w_i to be non-negative and normalized.

Learning Ensembles: Bagging

- The easiest ensemble learning technique is bagging, or bootstrap aggregating.
- In this approach, to learn each ensemble member i , you draw a different random sample of N_B training instances from the N training instances with replacement and use it to learn f_{θ_i} .
- The weights w_i are set to $w_i = 1/M$.
- Bagging typically results in ensemble members that have similar error levels, but make uncorrelated errors.
- Making predictions by averaging or voting using the whole ensemble then typically reduces the error.
- A common instance of this approach is the application of bagging to decision trees, which is called a *random forest*.

Example: Bagged Decision Trees



Learning Ensembles: Stacking

- Stacking is a common approach when constructing a heterogenous ensemble where every member is a different type of model.
- Here you simply learn each model as usual on the full training data set.
- Once the base models f_{θ_i} are learned, they can be used to create a new feature space $\phi(\mathbf{x}) = [f_{\theta_1}(\mathbf{x}), \dots, f_{\theta_M}(\mathbf{x})]$.
- We can then learn a second level linear model in the feature space as a mechanism for learning the ensemble weights w_i .
- This can result in better performing ensembles than using uniform weights when some members of the ensemble perform much better than others.

Learning Ensembles: Boosting

- Boosting is the most complex ensemble formation approach.
- It involves iteratively adding members to an ensemble to improve performance in parts of the feature space where the current ensemble produces high loss predictions.
- Gradient boosting a popular method with gradient boosted decision trees being the most popular model/algorithm combination and XGBoost a popular implementation.

Gradient Boosting Algorithm

Input: training set $\{(x_i, y_i)\}_{i=1}^n$, a differentiable loss function $L(y, F(x))$, number of iterations M .

Algorithm:

1. Initialize model with a constant value:

$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma).$$

2. For $m = 1$ to M :

1. Compute so-called *pseudo-residuals*:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)} \quad \text{for } i = 1, \dots, n.$$

2. Fit a base learner (or weak learner, e.g. tree) closed under scaling $h_m(x)$ to pseudo-residuals, i.e. train it using the training set $\{(x_i, r_{im})\}_{i=1}^n$.
 3. Compute multiplier γ_m by solving the following one-dimensional optimization problem:

$$\gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)).$$

4. Update the model:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x).$$

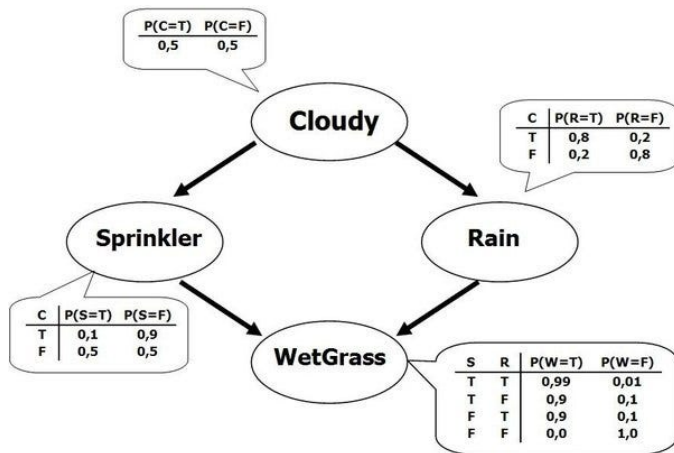
3. Output $F_M(x)$.

Bayesian Networks

- Bayesian networks are a generalization of the structured distributions we covered previously including products of marginals, mixture models, and factor analysis.
- They are defined via a directed acyclic graph $\mathcal{G}$ where the nodes represent random variables and the edges represent direct probabilistic dependence.
- Let pa_i represent the indices of parents of random variable X_i in the graph G and $\mathbf{X}_{pa_i}$ be the corresponding sub-vector of random variables.
- The joint distribution of $\mathbf{X} = [X_1, \dots, X_D]$ is given by:

$$P_{\theta}(\mathbf{X} = \mathbf{x}) = \prod_{d=1}^D P_{\theta}(X_d = x_d | \mathbf{X}_{pa_d} = \mathbf{x}_{pa_d})$$

Example: Sprinkler Network



Example: Bayesian Networks and Latent Variable Models

Product of Marginals



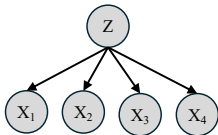
$$P(\mathbf{X} = \mathbf{x}) = \prod_{d=1}^D P(X_d = x_d)$$

General Latent Variable Model



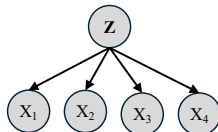
$$P(\mathbf{X} = \mathbf{x}, \mathbf{Z} = \mathbf{z}) = P(\mathbf{z} = \mathbf{z})P(\mathbf{X} = \mathbf{x}|\mathbf{Z} = \mathbf{z})$$

Mixture Model



$$P(\mathbf{X} = \mathbf{x}, \mathbf{Z} = \mathbf{z}) = P(\mathbf{Z} = \mathbf{z}) \prod_{d=1}^D P(X_d = x_d | \mathbf{Z} = \mathbf{z})$$

Factor Analysis



$$P(\mathbf{X} = \mathbf{x}, \mathbf{Z} = \mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}) \prod_{d=1}^D \mathcal{N}(x_d; \mathbf{W}_d \mathbf{z}, \Psi_{dd})$$

Restricted Boltzmann Machines

- RBMs are closely related to the factor analysis model family, Ising models from statistical physics, and Markov Random Fields.
- In the classical RBM, $\mathbf{h} \in \{0, 1\}^K$ is a vector of hidden units while $\mathbf{x} \in \{0, 1\}^D$ is a vector of observed values.
- The model parameterizes $P(\mathbf{X} = \mathbf{x}, \mathbf{H} = \mathbf{h})$.
- The model was used to enable a form of generative pre-training that, for a brief period, appeared to be critical to initializing deep neural network models.
- This is the contribution from Geoff Hinton that was cited in the 2024 Nobel Prize in Physics.

Restricted Boltzmann Machines

- Instead of being composed of two locally normalized probability distributions (e.g., $P(X|Z)$ and $P(Z)$), they are constructed from a joint *energy function* that requires explicit normalization. These models are also called energy-based models (EBMs).

$$E_W^k(\mathbf{x}, h_k) = - \left(\sum_{d=1}^D W_{dk}^P x_d h_k + W_k^B h_k \right)$$

$$E_W(\mathbf{x}, \mathbf{h}) = \sum_{k=1}^K E_W^k(\mathbf{x}, h_k)$$

$$P_W(\mathbf{X} = \mathbf{x}, \mathbf{H} = \mathbf{h}) = \frac{\exp(-E_W(\mathbf{x}, \mathbf{h}))}{\sum_{\mathbf{x}'} \sum_{\mathbf{h}'} \exp(-E_W(\mathbf{x}', \mathbf{h}'))}$$

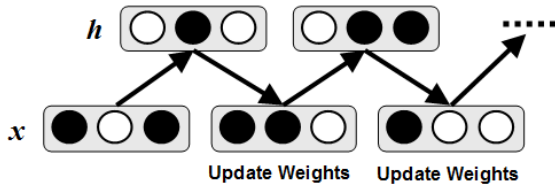
Restricted Boltzmann Machines: Stochastic MLE

- The RBM family can be learned using marginal likelihood maximization, but sampling is required to efficiently approximate marginal likelihood gradients. The result is a stochastic maximum marginal likelihood algorithm.

$$\mathcal{L}(\mathcal{D}, \mathbf{W}) = \sum_{n=1}^N \sum_{\mathbf{h} \in \mathcal{H}} P_{\mathbf{W}}(\mathbf{X} = \mathbf{x}_n, \mathbf{H} = \mathbf{h})$$
$$\frac{\partial \mathcal{L}(\mathcal{D}, \mathbf{W})}{\partial W_{dk}^P} = \sum_{n=1}^N x_{dn} P(H_k = 1 | \mathbf{x}_n) - \frac{N}{S} \sum_{s=1}^S \hat{x}_{ds} P(H_k = 1 | \hat{\mathbf{x}}_s)$$
$$\hat{\mathbf{x}}_s \sim P_{\mathbf{W}}(\mathbf{X})$$

Restricted Boltzmann Machines: Stochastic MLE

- The sampling procedure alternates between sampling $\hat{\mathbf{x}}$ and $\hat{\mathbf{h}}$ using the structure shown below. Only the $\hat{\mathbf{x}}$ samples are used for learning.



Learning as Optimization

- In both the MLE and ERM frameworks, we have tried to identify the optimal parameters θ of a probability distribution or prediction function given a sample of data $\mathcal{D}$.
- However, when the amount of data is low relative to the number of parameters, such approaches tend to overfit the available data and perform poorly.
- The optimization approach to dealing with this problem is to apply regularization or penalization.

Parameter Uncertainty

- An alternative view on this situation is that based on a small data set $\mathcal{D}$, there can be substantial *uncertainty* about the optimal values of the unknown parameters θ .
- This view motivates the idea of treating the unknown parameters θ as an *additional set of random variables*.
- Instead of trying to find the best single choice of the parameters θ given the data, we treat the parameters θ as an extra set of latent variables and marginalize over them.
- Any probabilistic model (including neural network models) can in principle be learned using Bayesian methods, although the require computation can be very challenging for large models.

Bayesian Supervised Learning

- Parameters: $\theta \in \mathbb{R}^K$
- Prior: $P(\theta)$
- Data: $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{1:N}$
- Conditional Likelihood: $P(\mathcal{D}|\theta) = \prod_{n=1}^N P(Y_n = y_n | \mathbf{X}_n = \mathbf{x}_n, \theta)$
- Joint: $P(\mathcal{D}, \theta) = P(\mathcal{D}|\theta)P(\theta)$
- Parameter Posterior: $P(\theta|\mathcal{D}) = \frac{P(\mathcal{D}, \theta)}{P(\mathcal{D})} = \frac{P(\mathcal{D}|\theta)P(\theta)}{\int P(\mathcal{D}|\theta)P(\theta)d\theta}$
- Posterior Predictive Distribution:
$$P(y|\mathbf{x}, \mathcal{D}) = \int P(y|\mathbf{x}, \theta)P(\theta|\mathcal{D})d\theta$$

Example: MCMC For Bayesian Linear Regression

